

TDV 2215 Terminal Self-Test Debug Guide

Overview

This guide provides detailed breakpoints and validation steps to debug the TDV 2215 terminal self-test sequence. Use this to identify exactly where your emulator fails during the boot process.

Pre-Test Setup Requirements

Memory Map Verification

Ensure your emulator implements the correct memory layout:

```
0x0000-0x1FFF: RAM (8KB) - Read/Write
0x2000-0x3FFF: ROM (8KB) - Read Only (Main firmware)
0x4000-0x47FF: ROM/RAM - Implementation dependent
0x5800-0x5FFF: Static RAM - System variables
0x6000-0x7FFF: Display RAM - Video memory
0x8000-0x8001: Optional ROM check area
```

I/O Port Requirements

```
0x60: Control Port 2 (Status/Control)
Z80SIO Ports: Implementation dependent
```

Phase 1: Boot Entry & Power-On Delay

Breakpoint 1.1: Boot Entry

Address: `0x01E9` (terminal_boot_sequence) **Action:** Verify execution starts here **Test:**

- Interrupts should be disabled
- Registers in unknown state (normal)

Breakpoint 1.2: Power-On Delay Loop Entry

Address: `0x01F0` **Action:** Check delay loop initialization **Test:**

- Register with 0x3000 (12,288) should be loaded
- Memory location `boot_progress_marker` (0x5FFD) should be uninitialized

Breakpoint 1.3: Power-On Delay Loop Exit

Address: `0x0202` (after delay loop) **Action:** Verify delay completed **Test:**

- Delay counter should be 0x0000
 - Approximately 12,288 CPU cycles should have elapsed
-

Phase 2: ROM Detection Test

Breakpoint 2.1: ROM Check Start

Address: `0x0202` **Action:** ROM presence test at 0x8000 **Test:**

```
Read bytes at 0x8000 and 0x8001
Calculate: (byte_8000 + byte_8001) & 0xFF
Expected: Result should NOT be 0x00 with carry set
If failure: Sets boot_progress_marker = 0x234 and halts
```

Breakpoint 2.2: ROM Check Failure Path

Address: `0x0208` (if ROM check fails) **Action:** System halt condition **Test:**

- `boot_progress_marker` (0x5FFD) = 0x234
 - System will initialize variables then halt
 - **EMULATOR ISSUE:** Check if 0x8000-0x8001 area returns proper values
-

Phase 3: Cold vs Warm Boot Detection

Breakpoint 3.1: System Status Check

Address: `0x0215` **Action:** Check system_status_word for boot type **Test:**

```
Read system_status_word (0x5E0C) - 16-bit value
High byte + Low byte calculation
If result != 0x00: Cold boot (run hardware tests)
If result == 0x00: Warm boot (skip tests)
Expected for first boot: Cold boot path
```

Breakpoint 3.2: Hardware Test Decision

Address: `0x0220` **Action:** Determine if hardware tests will run **Test:**

- If taking branch: Hardware tests will run
 - `boot_progress_marker` (0x5FFD) = 0x24A
 - **COMMON FAILURE POINT:** Many emulators fail here due to uninitialized memory
-

Phase 4: Hardware Power-On Test (POST)

Breakpoint 4.1: POST Entry

Address: `0x0AAB` (hardware_power_on_test) **Action:** Begin comprehensive hardware testing **Test:**

- `boot_progress_marker` (0x5FFD) should be 0x24A
 - Carry flag cleared (no previous errors)
-

Phase 4A: Memory Test

Breakpoint 4A.1: Memory Test Entry

Address: `0x0B8E` (memory_test) **Action:** RAM integrity test begins **Test:**

- HL register pair points to RAM start address
- Prepare for address-in-address pattern test

Breakpoint 4A.2: Memory Test Pattern Write Phase 1

Address: `0x0B94` (inside write loop) **Action:** Writing low-byte address patterns **Test:**

```
For each address X in RAM:  
    Write (X & 0xFF) to address X  
    Verify write completed successfully
```

Breakpoint 4A.3: Memory Test Verify Phase 1

Address: `0x0BA4` (inside verify loop) **Action:** Verifying low-byte patterns **Test:**

```
For each address X in RAM:  
    Read value from address X  
    Compare with (X & 0xFF)  
    If mismatch: Set carry flag and exit
```

Breakpoint 4A.4: Memory Test Pattern Write Phase 2

Address: `0x0BB0` (high byte write) **Action:** Writing high-byte address patterns **Test:**

```
For each address X in RAM:  
    Write ((X >> 8) & 0xFF) to address X  
    Verify write completed successfully
```

Breakpoint 4A.5: Memory Test Verify Phase 2

Address: `0x0BC0` (high byte verify) **Action:** Verifying high-byte patterns **Test:**

```
For each address X in RAM:
  Read value from address X
  Compare with ((X >> 8) & 0xFF)
  If mismatch: Set carry flag and exit
```

Breakpoint 4A.6: Memory Test Completion

Address: `0x0BC4` (LAB_ram_0bc4) **Action:** Memory test result processing **Test:**

- Carry flag clear: Memory test passed
 - Carry flag set: Memory test failed
 - **EMULATOR ISSUE:** Check RAM read/write functionality
-

Phase 4B: Display Memory Test

Breakpoint 4B.1: Display Memory Test Entry

Address: `0x0BE8` (initialize_display_memory) **Action:** Video RAM test begins **Test:**

- Display memory range: 0x6000-0x7FFF (8KB)
- Pattern: Incrementing bytes (0x00, 0x01, 0x02...)

Breakpoint 4B.2: Display Write Pattern

Address: `0x0BF8` (inside display write loop) **Action:** Writing incremental pattern to video RAM **Test:**

```
Pattern write to 0x6000-0x7FFF:
Address 0x6000 = 0x00
Address 0x6001 = 0x01
Address 0x6002 = 0x02
... continuing to 0x7FFF
```

Breakpoint 4B.3: Display Verify Pattern

Address: `0x0C18` (inside display verify loop) **Action:** Verifying display memory pattern **Test:**

```
For each address X in 0x6000-0x7FFF:
  Read value from address X
  Compare with expected pattern
  Check both full byte and nibble (4-bit) values
```

Breakpoint 4B.4: Display Test Success

Address: `0x0C30` (display test success path) **Action:** Display memory test passed **Test:**

- Carry flag should be clear
- All display memory verified

Breakpoint 4B.5: Display Test Failure

Address: `0x0C34` (display test failure path) **Action:** Display memory test failed **Test:**

- Carry flag should be set
 - **EMULATOR ISSUE:** Check display memory implementation
-

Phase 4C: Z80SIO Serial Port Test (CRITICAL FAILURE POINT)

Breakpoint 4C.1: Serial Test Entry

Address: `0x0C38` (test_z80sio_ports) **Action:** Z80SIO interface testing begins **Test:**

- Interrupts disabled
- Serial ports initialized
- **NOTE:** Code comment says "This is the place it fails right now!!"

Breakpoint 4C.2: Host Serial Port Init

Address: `0x0C48` (after init_host_serial_port call) **Action:** Channel A (host port) initialized **Test:**

- Z80SIO Channel A should be configured
- `statusport2_flag` = 1 (test mode)

Breakpoint 4C.3: DTR/RTS Signal Test

Address: `0x0C58` (DTR/RTS testing) **Action:** Testing control signal lines **Test:**

Test sequence:

1. Clear DTR/RTS (parameter 0x00)
2. Set DTR (parameter 0x80)
3. Set RTS (parameter 0x02)

Each should complete without carry flag set

Breakpoint 4C.4: Critical Loopback Test Setup

Address: `0x0C68` (before pattern test) **Action:** Prepare for loopback pattern test **Test:**

- `counter_init_0` = 0
- About to send test pattern 0x55

Breakpoint 4C.5: First Pattern Test (0x55)

Address: `0x0C70` (do_something_FAILS__ call) **Action:** Send test pattern 0x55 (01010101 binary) **Test:**

Expected: Function returns 0x55

If failure: Returns different value

Common issue: Z80SI0 not properly emulated

Breakpoint 4C.6: Second Pattern Test (0x2A)

Address: `0x0C78` (second pattern test) **Action:** Send test pattern 0x2A (00101010 binary) **Test:**

Expected: Function returns 0x2A (ASCII '*')

If failure: Returns different value

This tests complementary bit pattern to 0x55

Breakpoint 4C.7: Serial Test Error Paths

Address: `0x0CB1` (error handling) **Action:** Process serial test results **Test:**

Error codes in register A:

0x12: DTR/RTS signal failure

0x13: Pattern transmission general failure

0x14: Pattern loopback failure

Success: No error code, carry flag clear

Breakpoint 4C.8: Serial Test Cleanup

Address: `0x0CC8` (test cleanup) **Action:** Restore normal serial operation **Test:**

- Interrupts re-enabled
 - `statusport2_flag` = 0 (normal mode)
 - Test results processed
-

Phase 4D: NVRAM Test

Breakpoint 4D.1: NVRAM Test Entry

Address: `0x0AEA` (NVRAM test function) **Action:** Non-volatile memory test begins **Test:**

- `nvr_test_address_pointer` (0x5E03) = 0x50
- Test range: 0x50-0x5F (16 bytes)

Breakpoint 4D.2: NVRAM Write Test

Address: `0x0AF8` (write test) **Action:** Test NVRAM write capability **Test:**

Write test pattern to NVRAM

Expected return: `0x80` (write success)

If failure: Different return value

Breakpoint 4D.3: NVRAM Address Loop

Address: `0x0B08` (address increment loop) **Action:** Test each NVRAM location **Test:**

For addresses `0x50` through `0x5F`:

Write test pattern

Read back and verify

Increment `nvrām_test_address_pointer`

Breakpoint 4D.4: NVRAM Checksum Validation

Address: `0x0B38` (checksum check) **Action:** Verify NVRAM data integrity **Test:**

- `nvrām_checksum_accumulator` (`0x5E01`) should match expected
- If `BYTE_ram_5e01` != `0xAA`: Checksum failure

Breakpoint 4D.5: NVRAM Test Results

Address: `0x0B5F` (test completion) **Action:** Process NVRAM test results **Test:**

Success: Carry flag clear, continue

Failure codes:

`0x19`: Checksum failure

`0x1A`: NVRAM not responding

`0xB0+address`: Specific location failure

Phase 4E: NVRAM Configuration Load

Breakpoint 4E.1: Config Load Entry

Address: `0x0AD8` (`load_nvrām_config`) **Action:** Load saved configuration **Test:**

- Uses same function as NVRAM test
- Loads terminal settings from NVRAM
- Falls back to defaults if corrupted

Phase 5: System Initialization Completion

Breakpoint 5.1: Variable Initialization

Address: `0x024D` (after hardware tests) **Action:** Initialize system variables **Test:**

- `boot_progress_marker` (`0x5FFD`) = `0x24D`
- System variables setup
- Interrupt vectors configured

Breakpoint 5.2: Display Startup Message

Address: `0x0237` (if display message branch taken) **Action:** Show startup message to user **Test:**

- Terminal model string displayed
- Keyboard initialization command sent (`0x18`)

Breakpoint 5.3: System Ready

Address: `0x0269` (system ready) **Action:** Mark system operational **Test:**

- `system_status_word` (`0x5E0C`) = `0x8080` (warm boot marker)
- Hardware extensions called
- Ready for main loop

Breakpoint 5.4: Main Loop Entry

Address: `0x2002` (`terminal_main_loop`) **Action:** Enter operational mode **Test:**

- All self-tests completed successfully
- System ready for terminal operation
- **SUCCESS POINT:** Self-test sequence completed

Debug Strategy by Common Failure Points

1. Emulator Fails Immediately

Likely Issue: Memory map problems **Check:**

- ROM at `0x2000-0x3FFF` readable
- RAM at `0x0000-0x1FFF` read/write
- Execution starts at `0x01E9`

2. Fails During Memory Test

Likely Issue: RAM implementation **Check:**

- All RAM addresses writable/readable
- Address decoding correct
- No memory overlap issues

3. Fails During Display Test

Likely Issue: Display memory not implemented **Check:**

- 0x6000-0x7FFF range accessible
- Video memory separate from main RAM
- Display memory read/write functionality

4. Fails During Serial Test (Most Common)

Likely Issue: Z80SIO not emulated or incomplete **Check:**

- Z80SIO register implementation
- DTR/RTS signal handling
- Loopback capability for test patterns
- Status register proper values

5. Fails During NVRAM Test

Likely Issue: NVRAM/EAROM not implemented

Check:

- Non-volatile memory at proper addresses
- Write/read capability
- Checksum calculation support

Quick Test Procedure

1. **Set breakpoint at 0x01E9** - Verify boot starts
2. **Set breakpoint at 0x0220** - Check cold/warm boot detection
3. **Set breakpoint at 0x0BC4** - Verify memory test passes
4. **Set breakpoint at 0x0C34** - Check display test result
5. **Set breakpoint at 0x0C68** - Monitor serial test (likely failure point)
6. **Set breakpoint at 0x2002** - Confirm successful completion

Memory Watches During Debug

0x5FFD (boot_progress_marker) - Track boot progress
0x5E0C (system_status_word) - Cold/warm boot status
0x5E03 (nvram_test_address_pointer) - NVRAM test progress
0x5D52 (z80sio_channel_a_test_status) - Serial test results
Carry Flag - Test pass/fail indicator

This systematic approach will help you identify exactly where your emulator's self-test sequence fails and what component needs attention.